김명성

총 경력 5년 11개월

레거시를 무중단으로 전환하고, 판단의 근거를 RFC로 남겨 팀의 표준으로 정착시키는 프론트엔드 엔지니어

maeng2418@gmail.com 010 8663 2663 maeng2418.github.io github.com/maeng2418 maeng2418.github.io/portfolio

핵심 성과

FE

- **리액트 전환율 2022년 0% → 2026년 98.9%**
  Vue/Nuxt 레거시의 React 점진 전환
- **빌드 589.24s → 캐시 적중 시 1.10s**
  증분 빌드 도입
- **신규 디자인 컴포넌트 구현 및 VRT 적용**
  파트너 시스템 디자인 시스템 NDSP 컴포넌트 개발

AI 인프라

- **플러그인 8종 개발 및 개발생산성 향상**
  사내 Claude 플러그인 마켓플레이스(stay-claude-plugins) 구축
- **자동 PR 리뷰봇 개발 및 도입 운영**
  Claude Agent SDK 기반 PR 리뷰 봇 pr-reviewer 구축
- **도메인 지식 시스템을 LLM-Wiki 구조로 재설계**
  LLM을 활용한 도메인 Wiki 생성

경력

놀유니버스

2022.09 - 재직 중

Stay Supply FE팀 · FE Engineer

FE

Vue/Nuxt → React 점진 전환

문제	React 경력으로 입사해 Vue/Nuxt 레거시 앱 3종에 합류. 레거시 7레포 543건과 신규 10레포가 병존하는 상태에서 FE 3명이 함께 전환해야 했음
판단	전환 절차를 퍼블리싱 → API → TC → nginx 라우트 전환 → Vue 코드 제거 순서로 정함
실행	2023년 호텔 할인관리·요금제 React 전환(5개 repo)과 QA 결함 30건 전량 처리(2023.06), 2024년 전환 9차 수행(Jira 12건, PR 19건)
결과	리액트 전환율 2022년 0% → 2023년 30.6% → 2024년 77.7% → 2025년 93.5% → 2026년 98.9%. 팀 단위 전환율은 2024년 78/112 (69.6%)로 팀 성과다.

빌드·테스트 기반 정비

문제	테스트 코드가 전무했음. 모노레포에서 코드를 직접 참조 방식으로 임포트해 빌드된 결과물이 아닌 원본 코드를 가져왔고, 서비스 배포 때마다 그 코드를 함께 빌드해 빌드가 느렸음
판단	Turborepo는 빌드 결과물을 import하는 구조에 효과적이라, 소스 코드를 직접 참조하는 이 모노레포에는 적용하지 않고 증분 빌드로 해소하기로 함. E2E 테스트는 헤드리스 브라우저를 지원해 CI/CD 환경에 쓰기 쉽고 codegen 같은 생성 도구가 있어 테스트를 만들기 쉬운 Playwright를 택함
실행	TS 증분 빌드·순환 의존 제거, Playwright E2E 도입, devtools + MSW 코드 생성기 구축
결과	빌드 589.24s → 캐시 적중 시 1.10s, E2E 테스트 코드 신규 2,833라인

클라이언트 로그 계측 DevTools

문제	로그 정의 304건이 앱별 생성 코드에 흩어져 있어, 코드 밖에서 확인할 방법이 없었음
판단	확인 비용을 화면 위로 옮기기로 함 — 인스펙터를 vanilla DOM으로 만들어 MUI·NDSP 두 앱군에서 함께 쓰고, 수집 동작 변경은 파일럿 화면 한 곳으로 한정
실행	계측 API(clientLogTag / useClientLogPage), DevTools(카탈로그·인스펙터·txtpb 생성·자동 발화), 스크린샷 자동 캡처, infra-protobuf 확장 — 대상: 로그 정의 304건, 로그 센더 호출 파일 174개
결과	결과 계측을 후속 과제 근거로 활용

해외 파트너센트럴

문제	도메인이 복잡해질수록 기존의 기능 위주 아키텍처로는 유지보수가 어려웠음
판단	공통 정책을 화면이 아니라 하위 레이어로 내리고 화면은 조합만 하도록 FSD 레이어를 분리
실행	착수 단계에서 steiger FSD lint·useControllerGroup 공통 훅 도입, NOL Partner Central MVP의 캘린더·Rate Query 가상 스크롤 구현. 컨벤션 전면 적용은 Stacked PR 6단계로 나눠 진행
결과	정찰 시점 컨벤션 준수율 75~80%였던 코드를 전수 정규화

제휴입점관리자 기술 스택 전환 (리드)

문제 Nuxt 2 기반이라 지원 중단 문제가 있었고, 서버 관리 부담과 레거시 코드가 많았음

판단 [RFC] 제휴입점 관리자 React 전환 전략(대안 11개 대조)과 [TechSpec] Docker/Kubernetes 및 path 분기 문서 작성

실행 FSD, Module Federation, S3 + CloudFront, CI/CD, Dockerfile, EKS manifest 구축, 클라이언트 번들에 노출되던 Sentry auth token 제거

결과 SSR에서 CSR로 바뀌어 더 이상 서버 관리가 필요 없고 CDN 방식으로 운영. Nuxt에서 React로 옮겨 개발 리소스를 줄이고 최신 라이브러리를 활용

NDSP 컴포넌트

문제 파트너 시스템 전체 디자인을 하나로 통일하고, MUI 등 외부 라이브러리 의존성을 없애야 했음

판단 MUI·antd의 인터페이스를 참조해 입력 필드 등의 값과 핸들러를 일관되게 맞춤

실행 NDSP 신규 컴포넌트 2025년 8종, 2026년 5종 추가, Fieldset 컴포넌트 VRT 등록

결과 라이브러리를 설치한 서비스를 추적해 사용처가 점차 늘고 있음을 확인. VRT로 CI/CD 테스트 안정성 확보

구조 개선

문제 판매 잠금 기준이 11개 파일에 흩어져 있어 정책 하나를 바꾸려면 모든 사용처를 찾아 고쳐야 했음

판단 채널별 규칙을 코드 분기가 아닌 정책 선언으로 분리

실행 흩어진 기준을 선언 한 곳으로 모으고, 사용처 코드는 수정하지 않음

결과 정책 변경 지점 11개 파일 → 1곳

요금제 화면 개선

문제 API 개발이 끝날 때까지 FE 작업을 시작하지 못해 FE·BE 작업이 순차로 이어졌음

판단 API 개발이 끝나기 전에 FE·BE가 스펙을 먼저 맞춘 뒤 동시에 병렬로 작업하기로 함

실행 CM 할인 요금제 작업에서 백엔드보다 먼저 Mock API 작성, prop drilling을 FormProvider로 정리하고 무한 리렌더 수정

결과 리렌더 문제 해소

파트너센터 운영 개선

문제 숫자 입력 컴포넌트 통합이 두 번 revert 됨

판단 테스트 케이스를 먼저 작성한 뒤 재적용

실행 CHANGELOG.md 4개 repo 도입, 폰트를 외부 CDN에서 내부 리소스로 이전

결과 다양한 테스트 케이스로 이슈를 최소화하고 여러 상황에 대응

어드민·연동·프로모션 화면

문제 이전에는 CM 연동과 프로모션을 지원하지 않아 다양한 상품을 판매할 수 없었음

판단 파트너센트럴 화면이 내부 관리자용 파트너센트럴관리자 화면과 비슷하거나 같아, 관리자 화면을 먼저 개발하고 재사용

실행 외부 사용자 Admin 12개 화면, CM 연동 잠금 정책 3단계 확장, 프로모션 화면 9개·API 6개

AI 인프라

AI 개발 인프라 — 사내 Claude 플러그인 마켓플레이스

문제 개발 도구가 모노레포에 종속돼 있었고 개인 자산과 팀 자산의 경계가 불분명했으며, 필수 규칙이 누락될 위험이 있었음. 당시 팀원들의 AI 인지도와 학습도가 낮아 AI 활용을 높일 필요도 있었음

판단 대안 9개를 대조해 레포 종속 자산과 공용 자산을 나누는 2축 구조를 채택하고, Claude Plugin 구성 및 배포 전략을 문서화(2026.04)

실행 플러그인 레포 stay-claude-plugins 분리, 레포별 플러그인, Atlassian MCP OAuth, 팀 statusline; 커맨드 create-pr v2·create-jira·/deploy·design-sync·feature-flag·publish-artifact; 마켓플레이스 manifest·캐시 경로 수정, CI/lint/format, GitHub rulesets 정렬

결과 플러그인 8종 배포, 소비 레포 5개에 적용

stay-context 지식 플러그인과 LLM-Wiki

문제 도메인이 많아, 도메인 지식이 사라지지 않도록 데이터를 쌓아 위키로 활용할 방법이 필요했음

판단 LLM-Wiki 전환에 앞서 대안 3종을 기각 근거와 함께 남김

실행 도메인 지식 플러그인 stay-context 구축, LLM-Wiki 전환과 대형 모노레포 지식 빌드 파이프라인. ADD/UPDATE/NOOP 판정 구조와 자기증식 방지 2계층 가드 설계

결과 SessionEnd 훅이 자동 생성한 PR 270건 중 54건 머지, 감사 점수 0.52 → 0.87, evaluator 93/85/90/92, 테스트 129건 통과

PR 리뷰 봇 pr-reviewer

문제 리뷰와 PR 승인이 늦어져 개발 속도가 떨어졌고, 결함은 사람보다 AI가 더 잘 찾아냈음

판단 팀에서 주로 쓰는 AI가 Claude라 Claude Agent SDK를 택함. 엔진과 콘솔은 실행 환경(Node·SDK와 Tauri 웹뷰)이 다르고, GitHub 토큰과 SDK 자격 증명을 엔진에만 두는 보안 경계가 필요하며, 엔진만 따로 실행(dry-run·테스트)할 수 있어야 하고, 한쪽 장애가 다른 쪽으로 번지지 않아야 해서 나누기로 함

실행 Claude Agent SDK 기반 PR 리뷰 봇 pr-reviewer 구축 — 콘솔과 엔진을 별도 앱으로 분리. 엔진은 Claude SDK로 PR을 리뷰하고 지적한 줄이 맞는지 검증한 뒤 gh로 게시하며 실행 기록을 SQLite에 남기고, 콘솔은 Tauri 데스크톱 앱으로 트레이·승인 말풍선·대시보드를 보여 주되 DB는 읽기만 하고 화면이 부르는 Rust 명령은 허용 목록에 있는 것만 통과

결과 Claude Agent SDK 기반 자동 PR 리뷰봇 pr-reviewer 개발 및 도입 운영

AI 컨벤션 자산

문제 한번 만든 스킬과 규칙이 어느 순간 레거시가 되지 않도록 계속 업데이트하고 관리할 방법이 필요했음

판단 규칙을 실행 계층으로 내려, 작업 시점에 자동으로 적용되게 하기로 함

실행 glob-scoped .claude/rules, 모노레포 CLAUDE.md 18개 감사, ADR → 스킬 전환, /project-sync와 도메인별 감사 에이전트; 2025년 Cursor·Claude Code 일상 도입과 팀 AI 도구 가이드 공유

결과 규칙이 바뀌면 스킬 한 곳만 고치면 됨

CI/CD 자동화

문제 여러 GitHub Actions를 수동으로 돌리고 Slack 알림 메시지도 직접 작성해야 하는 불편이 있었음

판단 피처플래그를 엑셀 시트로 관리하다 보니 오타 같은 실수가 잦고 이력 관리도 잘 안 돼, 피처플래그를 S3 기반으로 옮기기로 함

실행 GitHub Actions composite action 8종, swap 전 Slack 자동 알림, S3 기반 피처플래그 시스템과 플래그 13개 제거

결과 Launch Plan 33회 실행

린트·CI 게이트

문제 모노레포가 커질수록 로컬에서 돌리는 작업의 부담이 커지고 느려졌음

판단 반복 지적 사항은 린트 규칙으로, 빌드·타입 검증은 CI 게이트로 승격

실행 React hook deps ESLint 룰, dev/master 직접 push 차단, Sentry 로그 타이틀 정비, Auto-Release 자동화, PR 템플릿과 production push 보호(5개 repo), 리뷰 체크박스 merge-control Action, commitlint·cspell 도입

결과 신규 레포는 첫 PR로 세팅해 첫 커밋부터 규약이 강제되도록 함. 일부 기능은 GitHub Actions로 분리해 옮겨 시간을 줄임

AI 정책 분류기

문제 정책 문서가 여러 곳에 흩어져 있었고, 이를 하나의 스프레드시트로 자동 관리·유지보수하길 원했음

판단 누구나 흩어진 정책 문서 페이지에서 손쉽게 정책을 추출할 수 있도록 Chrome 확장 형태를 택함

실행 AI 정책 분류기를 Chrome 확장으로 개발

결과 Q4 공식 프로젝트로 진행

연도별 작성 PR · 타인 PR 리뷰: 2022년(입사 후) 45 · 22, 2023년 369(중복 제거) · 474, 2024년 404 · 798, 2025년 480 · 554

2026년(2026.10 기준) 작성 PR 913건, 그중 270건은 stay-context 자동 생성; 타인 PR 리뷰 752건

IGAWorks (Adpopcorn)

2021.07 - 2022.09

FE Engineer

애드팝콘 SSP 콘솔 신규 구축

문제 기존 Next.js SSR 구조가 콘솔 성격에 맞지 않았음 — SSR로 개발됐지만 레거시 코드가 많았고 SSR의 장점을 살린 코드도 아니었음

판단 CSR 쪽의 장점이 더 크다고 판단. 배포는 ECS에서 AWS Amplify로 변경

실행 Next.js 기반 기존 콘솔을 대체하는 신규 CRA 콘솔 구축(2022.04-2022.09), React-Query + Recoil 상태 관리, Recharts 대시보드

결과 커밋·푸시하면 Amplify가 자동으로 빌드·배포

콘픽

문제 서버에서 HTML 코드를 받아 렌더링하는 서비스라, 별도 FE 소스 코드보다는 서버 운영에 가까운 프로젝트였음

판단 서버 운영에 가까운 프로젝트라 TypeScript 지원이 강한 NestJS를 택함

실행 NestJS로 뉴스 콘텐츠 HTML 변환, 뉴스 템플릿 생성 클래스, 광고 삽입 구현(2022.08-2022.09), ECS + CodePipeline CI/CD 구성 (buildspec·appspec·taskdef 직접 작성)

결과 커밋·푸시하면 dev·prod CodePipeline 파이프라인이 자동으로 빌드·배포

애드팝콘 홈페이지

문제 회사 리브랜딩과 사업 분할로 새 페이지가 필요했음

판단 정적 사이트라 Gatsby를 택함

실행 Gatsby + Tailwind + i18n 기반 홈페이지 구현(2022.08-2022.09) — Monetize·Adserver·Offerwall 페이지, 문의하기 기능과 영문 페이지, GA

코드 품질 도구

문제 이전에는 코드 규칙이 따로 없어 코드가 무질서했음

판단 깃에 올라가기 전에 코드 안정성과 일관성을 확보하려고, 커밋 시점에 Husky + lint-staged로 검사를 강제함

실행 ESLint + Prettier + Husky + lint-staged 적용

결과 CI/CD 빌드 이슈가 줄고 코드 일관성이 확보돼 협업에 유리해짐

TypeScript

문제 자동 타입 변형(암묵적 형 변환) 이슈가 있었음

판단 타입 안정성을 확보하기 위해 TypeScript를 택함

실행 직접 시작한 레포는 첫 커밋부터 TypeScript로 작성

결과 애드팝콘 SSP 콘솔 레포 기준 ts/tsx 파일 625개, js 파일 0개

짐싸 (파트너 앱)

2020.12 - 2021.07

FE Developer

파트너 앱

문제 네이티브가 아니라 업데이트와 개발이 중단된 Cordova로 개발하고 있었음

판단 타입 안정성을 확보하고 협업에 유리하도록 TS 기반 새 프로젝트를 만들어 옮기기로 함

실행 Cordova 하이브리드 앱(Android) 개발, JS→TS 전환 착수 — CRA TS 템플릿 기반 프로젝트 생성·의존성 업데이트, 다크 모드, ESLint + Prettier + Husky + lint-staged 적용

결과 커밋할 때마다 lint-staged가 eslint --fix·prettier --write를 자동 실행. src는 js/jsx 422개·ts/tsx 31개로 TS 전환은 착수 단계

파트너 웹 v2 컴포넌트 시스템

문제 개발자마다 UI 설계가 제각각이었음

판단 통일된 구조로 일관성을 확보하고 컴포넌트를 재사용하려고 Atomic Design를, 테스트를 위해 Storybook를 택함

실행 Atomic Design 컴포넌트 시스템(atoms·molecules·organisms)과 Storybook 구축

결과 Storybook stories 파일 123개

디자인 2.0 리뉴얼

문제 다크 모드 지원이 필요했음

판단 목록에 보여 줄 아이템이 많아 로딩과 렌더링이 길고 버벅임도 있어, UI/UX와 성능을 개선하려고 Lazy-Loading을 적용하기로 함

실행 파트너 앱 디자인 2.0 리뉴얼 담당, Lazy-Loading 적용

결과 Lazy-Loading 적용으로 초기 로딩 개선

개인 프로젝트



claude-administer

Claude Administer (2026.10) — 스킬 · 커맨드 · 에이전트 · 훅 · 플러그인 · MCP 서버 · 마켓플레이스 · 규칙을 전역과 프로젝트에 걸쳐 하나의 목록으로 모아 주는 macOS 메뉴바 앱 · Electron · React · TypeScript · Vite

https://github.com/maeng2418/claude-administer



bookmark-pro

BookmarkPro (2025.07 - 2025.12) — 웹과 브라우저 확장프로그램에서 북마크를 효율적으로 관리할 수 있는 모던 북마크 관리 솔루션 — Turborepo 모노레포 구조 · Next.js · Supabase · shadcn/ui · Manifest V3

https://github.com/maeng2418/bookmark-pro



Maeng-Design

Maeng Design (2021.08 - 2022.03) — React UI 라이브러리 — npm 패키지 maeng-design, 문서 사이트 · React · TypeScript · Emotion · Storybook

https://maeng-design-docs.netlify.app/



SEOUL13_edition2

SEOUL13 Edition2 (2020.09 - 2021.06) — SEOUL13 회의실 예약 웹어플리케이션 Edition2 · React · TypeScript · Next.js · Jest · Express · MySQL · 팀 프로젝트

보유 스킬

프론트엔드	TypeScript · React · Next.js · Vue · Nuxt
상태·폼	React Query · Recoil · React Hook Form · Zod
UI·구조	MUI · Tailwind · Emotion · Storybook · FSD
빌드·테스트	Playwright · MSW · Module Federation
인프라·CI	GitHub Actions · Docker · Kubernetes · S3 · CloudFront · Sentry
AI 개발 도구	Claude Code · Claude Agent SDK · MCP · Cursor

작성 문서·표준

[RFC] 제휴입점 관리자 React 전환 전략 · [TechSpec] Docker/Kubernetes 및 path 분기 · [RFC] 해외숙소 폼·데이터 변환 컨벤션 (2026) · Claude Plugin 구성 및 배포 전략 (2026.04)

학력·활동

서울과학기술대학교 · 컴퓨터공학과 학사	2015.03 - 2021.02
우아한 테크 캠프 3기	2020.07 - 2020.08
철도경영정책연구소 연구원	2019.10 - 2020.06
DevelUp(구름) 1기	2019.09 - 2019.11
데이블 Blokee FE 인턴	2019.08 - 2019.09

maeng2418@gmail.com